

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #7 - Semana 8 – Sivarama - Capítulo 11
Procedimientos

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 09/09/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 231–253.

Abstract—*This paper explores key concepts of assembly language programming related to procedure management in the IA-32 architecture. It details the use of the stack for procedure invocation and parameter passing, highlighting the stack's LIFO nature and operations like push and pop. It explains basic stack instructions, the call and ret instructions for procedure control, and methods for parameter passing via registers and the stack. Additionally, the paper discusses stack cleanup techniques and preserving the caller's state, emphasizing the importance of saving and restoring registers. It concludes with an overview of the enter and leave instructions for managing stack frames.*

I. Introducción

En la programación en ensamblador, los procedimientos son unidades de código que facilitan la programación modular. En la arquitectura IA-32, el stack (pila) juega un papel crucial en la invocación y ejecución de procedimientos. Este capítulo detalla cómo usar el stack y las instrucciones de ensamblador asociadas, así como los métodos de paso de parámetros entre procedimientos.

II. Procedimientos en lenguaje ensamblador

Un procedimiento es una unidad de código que realiza una tarea específica y puede devolver uno o más resultados. En los lenguajes de alto nivel, como C, existen funciones y procedimientos, donde:

- Las funciones reciben argumentos, realizan cálculos, y devuelven un valor.
- Los procedimientos también reciben argumentos, pero pueden devolver cero o más resultados.

Existen dos métodos principales para pasar parámetros:

- Por valor: Se pasan los valores actuales de los argumentos a la función.
- Por referencia: Se pasan las direcciones de memoria de los argumentos, permitiendo que la función modifique los valores originales.

III. La pila (stack)

El stack es una estructura de datos de tipo LIFO (Last In, First Out), similar a una pila de platos en una cafetería. Las dos principales operaciones en un stack son:

- Push: Inserta un elemento en el stack.
- Pop: Elimina el elemento en la cima del stack.

En la arquitectura IA-32, el stack se implementa usando los registros SS (Segmento de Stack) y ESP (Puntero de Stack). El stack crece hacia direcciones de memoria más bajas, y las operaciones de push y pop modifican el valor de ESP para apuntar a la cima del stack.

IV. Instrucciones básicas de stack

Las instrucciones básicas para manipular el stack son:

- Push: Inserta datos en el stack. Ejemplo: push source.
- Pop: Elimina datos del stack. Ejemplo: pop destination.

Las instrucciones adicionales para manejar el stack incluyen:

- Pushfd/Popfd: Guardar y restaurar los registros de bandera.
- Pusha/Popa: Guardar y restaurar todos los registros de propósito general (excepto ESP).

V. Usos del stack

El stack se utiliza para:

- Almacenamiento temporal de datos: Permite almacenar datos temporalmente sin usar registros generales. Ejemplo: intercambiar valores entre variables.
- Transferencia de control del programa: Almacena el estado del programa al llamar y regresar de procedimientos.
- Paso de parámetros en llamadas a procedimientos: Utiliza el stack para pasar parámetros cuando el número de registros disponibles es insuficiente.

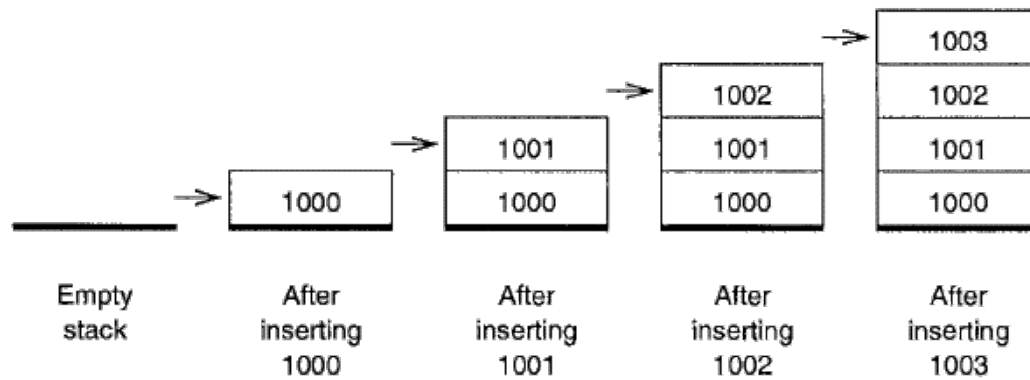


Figura 1. Representación de inserciones en el stack

VI. Instrucciones de procedimiento

Las instrucciones `call` y `ret` se utilizan para escribir procedimientos en lenguaje ensamblador.

Instrucción `call`:

- **Función:** Invoca un procedimiento.
- **Formato:** `call proc-name`, donde `proc-name` es el nombre del procedimiento a llamar.
- **Detalles:** El ensamblador reemplaza `proc-name` con el valor de desplazamiento de la primera instrucción del procedimiento llamado. El valor de desplazamiento en la instrucción `call` no es absoluto, sino un desplazamiento relativo en bytes desde la instrucción siguiente a `call`. El procesador almacena el valor del registro EIP (Instruction Pointer) en la pila antes de transferir el control al procedimiento llamado. El valor del desplazamiento se calcula como la diferencia entre las direcciones de las instrucciones y se codifica en el formato little-endian.

Instrucción `ret`:

- **Función:** Regresa el control desde el procedimiento llamado al procedimiento que hizo la llamada.
- **Detalles:** La dirección de retorno se obtiene de la pila. Durante la ejecución de `ret`, el registro EIP se actualiza con la dirección de retorno, y el puntero de pila (ESP) se ajusta para liberar el espacio usado.

VII. Paso de parámetros

El paso de parámetros en ensamblador puede hacerse mediante registros o la pila.

Método de registros:

En este método, los parámetros se colocan en los registros generales antes de llamar al procedimiento.

- **Ventajas:**
 - Conveniente para un número pequeño de parámetros.
 - Más rápido, ya que los parámetros están en los registros.
- **Desventajas:**
 - Limitación en el número de registros disponibles.
 - Los registros deben ser guardados y restaurados si se usan para otros fines, lo que puede añadir sobrecarga.

Método de pila:

En este método, los parámetros se colocan en la pila antes de la llamada.

- **Ventajas:**
 - Permite pasar más parámetros, ya que no está limitado por el número de registros disponibles.
- **Desventajas:**
 - La lectura de los parámetros desde la pila puede ser complicada, ya que los parámetros están anidados y el puntero de pila cambia.
 - Requiere manipulación adicional de la pila, y suele utilizarse el registro EBP como puntero de marco (frame pointer) para acceder a los parámetros en la pila, permitiendo una forma más estructurada de manipular los datos.

VIII. Limpieza de la pila

Se pueden usar dos métodos para limpiar la pila después de una llamada:

- La limpieza la realiza el procedimiento que hace la llamada.
- La limpieza la realiza el procedimiento llamado.

El segundo método es preferido para procedimientos con un número fijo de parámetros, ya que simplifica la limpieza al realizarla solo una vez en el procedimiento llamado.

IX. Preservación del estado del procedimiento llamador

Es crucial preservar el contenido de los registros durante una llamada a procedimiento para evitar cambios no deseados en el estado del programa. Los registros que se usan en el procedimiento llamado y que pueden ser modificados deben ser guardados temporalmente en la pila. El procedimiento llamado generalmente guarda los registros que usa y los restaura antes de regresar al procedimiento llamador, siguiendo los principios de diseño modular del programa.

Registros a guardar:

- **Qué guardar:** Los registros que el procedimiento llamador utiliza pero que el procedimiento llamado modifica.
- **Práctica común:** El procedimiento llamado guarda los registros que usa y los restaura antes de regresar al procedimiento llamador.

Uso de pusha:

- **Función:** La instrucción pusha guarda todos los registros generales en la pila.
- **Ventajas:** Puede ser útil para preservar el estado de todos los registros de propósito general.
- **Desventajas:** Introduce sobrecarga y no es recomendable si algunos registros se usan para retornar resultados, o si solo se necesitan guardar uno o dos registros.

X. Instrucciones enter y leave

Las instrucciones enter y leave facilitan la asignación y liberación del marco de pila al entrar y salir de un procedimiento. La instrucción enter asigna espacio para variables locales en el marco de pila, mientras que leave libera el marco al finalizar el procedimiento.